



ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

COURSE CODE: EEE 4308

COURSE NAME: DIGITAL ELECTRONICS LAB

PROJECT: SAP-1 Architecture based 8-bit Computer Design and Implementation.

Phase: D

GROUP NAME: Boolean Squad

GROUP MEMBERS:

1. IKRAM HOSSAIN (230021111)
2. NADMAN IBNE MAMUN (230021114)
3. FARHAN ISHRAQ (230021124)
4. SAMIN YASAR SAMI (230021138)
5. ZAVID ATAHAR SAMIN (230021155)
6. AGHA MAKINUN AMIN CHOWDHURY (230021232)
7. RAZEEN ABRAR RAHMAN (230021215)

OBJECTIVES:

The objective of **phase D** is to design and implement the following parts:

1. Output Unit
2. Integration with 8-bit bus

DESCRIPTION:

Output Unit:

Output Register:

The **Output Register** is an 8-bit storage module designed to latch results from the system bus and present them for user observation. Utilizing two **74LS173** registers, it captures data on the **Rising Edge** of the clock only when the **OI (Output In)** control signal is activated. To ensure hardware compatibility, a **74HC04 inverter** polarizes the high-level control signal into an active-low enable for the registers. Because the output enable pins are permanently grounded, the stored value is continuously displayed on the **000-007** pins, providing a stable visual readout of the computer's final computational output.

Integration with 8-bit bus

The **8-Bit System Bus** serves as the central communication backbone that links every functional module of your computer, enabling coordinated data transfer. To manage this shared pathway without electrical conflicts, the design utilizes **74LS245** (or similar) octal bus transceivers which act as tri-state buffers. These buffers ensure that a module only "talks" to the bus when its specific enable signal (like **EO** for ALU or **IO** for IR) is active, otherwise remaining in a high-impedance (Hi-Z) state to stay isolated. This parallel architecture allows 8-bit data—such as memory addresses, instructions, or computational results to move efficiently between source and destination registers during the precisely timed Fetch-Execute cycles.

Output Register

The **Output Register** is a non-bus-driving 8-bit storage unit that captures data from the system bus to present it to the user. It is implemented using two **74LS173** 4-bit D-type registers (U19 and U20).

- **Data Input (OI0-OI7):** These lines are connected to the 8-bit system bus.
- **Data Output (OO0-OO7):** These are the output lines that typically drive an LED bar or a 7-segment display driver. Unlike other registers, the output here is permanent (connected to ground via OE pins) so the user can always see the value.
- **Control Logic:** The register is controlled by the **OI (Output In)** signal. This signal passes through a **74HC04 hex inverter** (U1:A) before reaching the **active-low enable pins (E1 and E2)** of the 74LS173 chips. This ensures that when the control unit sends a HIGH signal for **OI**, the registers become active to load data.
- **Clock and Reset:** The module is synchronized with the system **CLK** and can be cleared using the **Reset** line connected to the Master Reset (MR) pins.

Function Table:

Reset (MR)	OI (Output In)	CLK	Data Bus (OI0-OI7)	Logic State	Output (OO0-OO7)
High (1)	X	X	X	Reset Mode	0000 0000
Low (0)	Low (0)	Trigger	D	Hold Mode	No Change (Last Value)
Low (0)	High (1)	Trigger	D	Load Mode	Updated to D
Low (0)	X	Low/High	X	No Clock	No Change

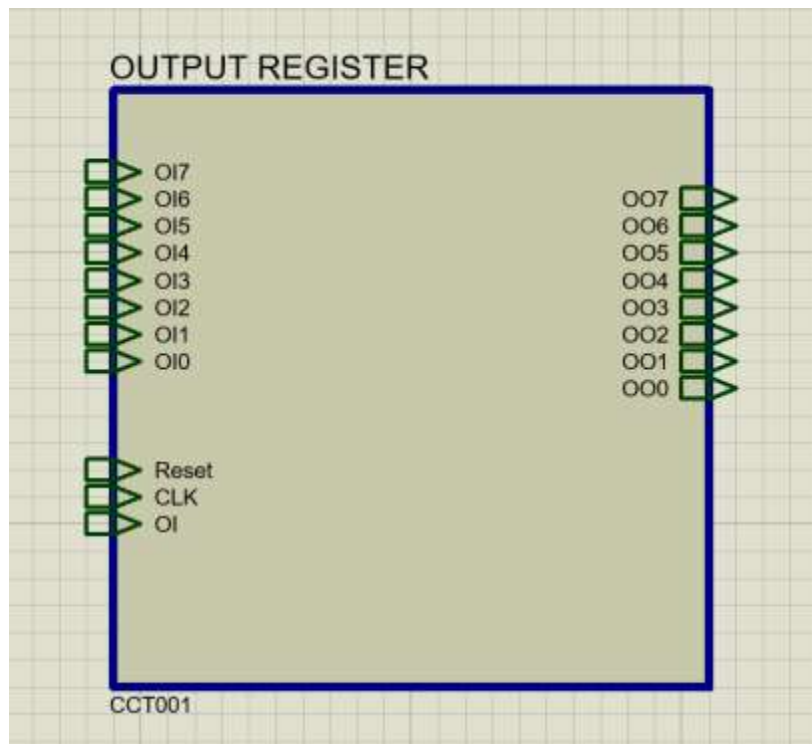
Operation Algorithm

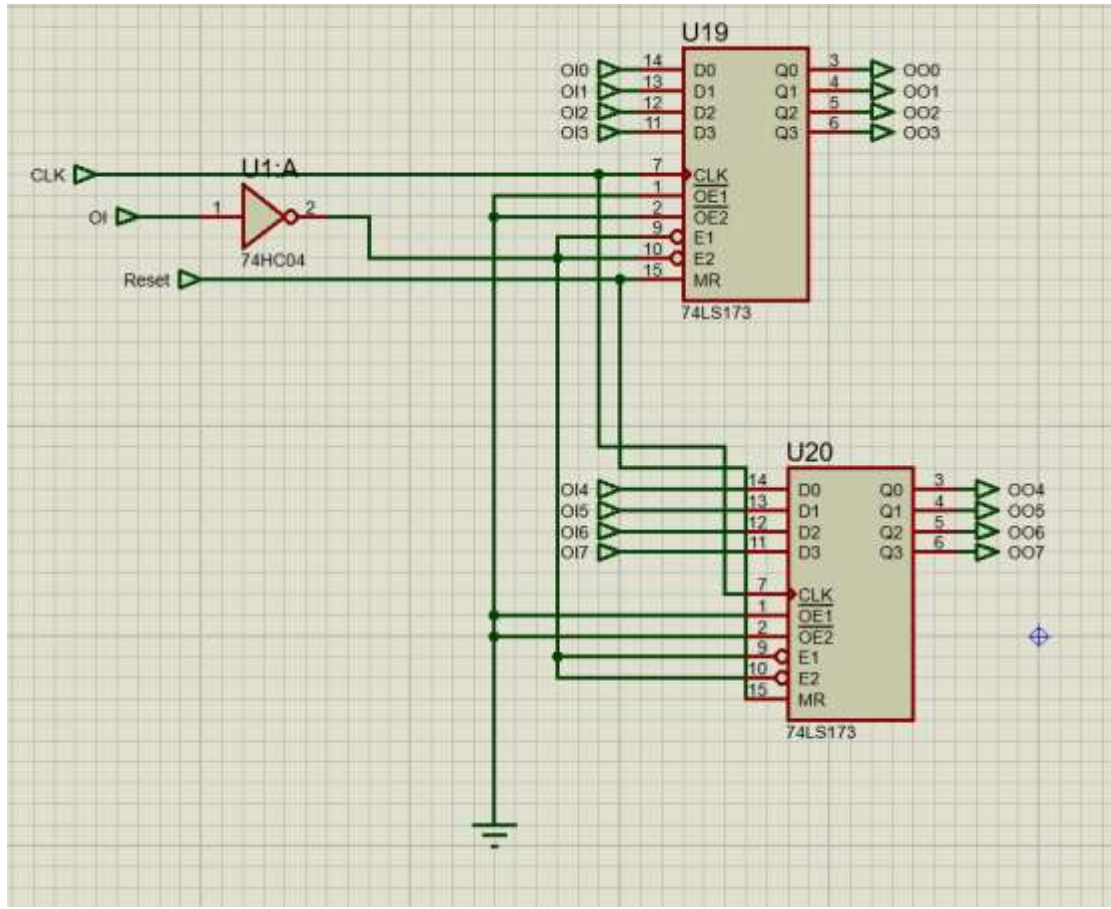
This algorithm describes the step-by-step logic the hardware follows to display a result:

1. **Idle State:** The **OI** signal is kept LOW (Logic 0). The inverter (U1:A) outputs a HIGH to the enable pins, causing the registers to ignore any data changes on the system bus.
2. **Enable Data Capture:** The Control Unit triggers the **OI** signal HIGH (Logic 1).

3. **Inversion:** The 74HC04 inverter flips the signal to LOW (Logic 0), enabling the E1/E2 pins of U19 and U20.
4. **Latch on Clock:** On the next **Rising Edge** of the system **CLK**, the 8-bit data currently on the bus (OI0-OI7) is latched into the internal flip-flops of the registers.
5. **Permanent Display:** Since the **Output Enable** pins are permanently tied to Ground, the latched 8-bit data is immediately and continuously visible on the output pins (**OO0-OO7**).
6. **Reset (Optional):** If the **Reset** line is pulled HIGH, the MR pins clear the registers, forcing the output to **0000 0000**.

Circuit Diagram:





Limitations:

The primary **limitation** of this output register design is its **volatility**, meaning all displayed data is immediately lost if the system is reset or powered down. Additionally, because the **Output Enable** pins are permanently grounded to provide a continuous display, the module is strictly a "one-way" destination and cannot write data back onto the system bus for reuse in other calculations. The circuit also lacks **Asynchronous Loading** capabilities, meaning it is entirely dependent on the system clock to update its state, which can lead to display lag if the clock speed is extremely low. Finally, without an onboard **Binary-to-Decimal decoder**, the output is restricted to raw binary format (via LEDs), making it less intuitive for users who require decimal or hexadecimal readouts.

Integration with 8-bit bus

BUS IMPLEMENTATION:

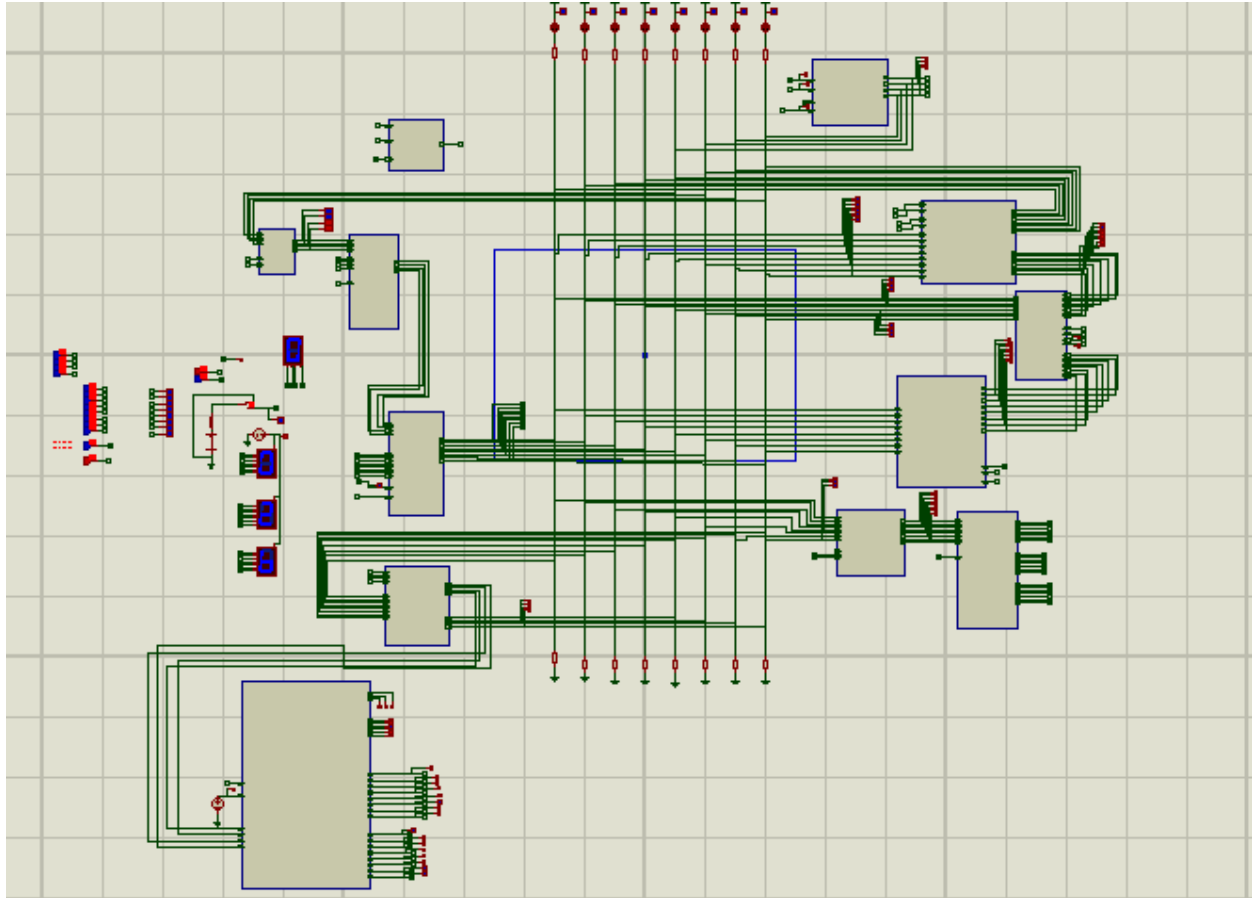


FIG: All components connected to the bus

In the SAP-1 architecture, the **Bus** is the central nervous system of the computer. It is a single **8-bit parallel communication path** that connects all major functional blocks (Registers, ALU, Memory, etc.).

Since only one component can "talk" (place data) on the bus at any given time to avoid short circuits (bus contention), each part uses **Tri-state Buffers** to connect or disconnect from the lines.

Bus Definition:

The bus is a collection of 8 wires labeled **D0 through D7**. It operates on the principle of **Time-Division Multiplexing**, where the Control Unit ensures that only one "Output Enable" signal is active during any specific T-state of the machine cycle.

2. Individual Part Implementations

The Accumulator (Register A)

- **Input:** Connected to the bus via D-type flip-flops. It loads data from the bus when the **LDA(Load A)** signal is low.
- **Output:** Connected to the bus through tri-state buffers. It places its stored byte onto the bus only when **EA (Enable A)** is active.
- **Internal Link:** It also has a hardwired (non-bus) connection directly to the ALU inputs.

The B Register

- **Input:** Loads data from the bus when **L_B** is active.
- **Output:** Unlike Register A, the B Register typically does **not** have an output to the bus. It exists solely to feed the second operand to the ALU logic gates.

The ALU (Arithmetic Logic Unit)

- **Input:** Receives operands directly from Register A and Register B (bypassing the bus for speed).
- **Output:** The result (Sum, Difference, AND, or OR) is sent to the bus through the **OR-merger gate array** and tri-state buffers. This output is gated onto the bus only when the **EU (Enable Unit)** signal is triggered.

Program Counter (PC)

- **Output:** The 4-bit address stored in the PC is placed onto the lower nibble of the bus (D0- D3) when **EP** is active, allowing the MAR to fetch the next instruction address.

Memory Address Register (MAR)

- **Input:** It "snaps" the address from the bus when **LM** is active. It does not have a bus output; its output is hardwired to the RAM address pins.

RAM (Random Access Memory)

- **Output:** When the **CE (Chip Enable)** signal is active, the RAM retrieves the byte at the address specified by the MAR and places it onto the 8-bit bus for the Instruction Register or Accumulator to read.

Instruction Register (IR)

- **Input:** Loads the instruction from the bus when **LI** is active.
- **Output:** The lower 4 bits (operand/address) are placed back onto the bus when **EI** is active, while the upper 4 bits (opcode) go directly to the Controller/Sequencer.

FEATURE (Modification of ALU):

For our feature, we added 3 new instructions to SAP-1. Two of them were to add a bitwise AND and a bitwise OR operation. To do this, we needed to modify the ALU. We did this by adding 3 ICs. These are:-

1. 74HC139 (2-to 4-line demultiplexers)
2. 8 two input AND gates
3. 8 two input OR gates

DESCRIPTION

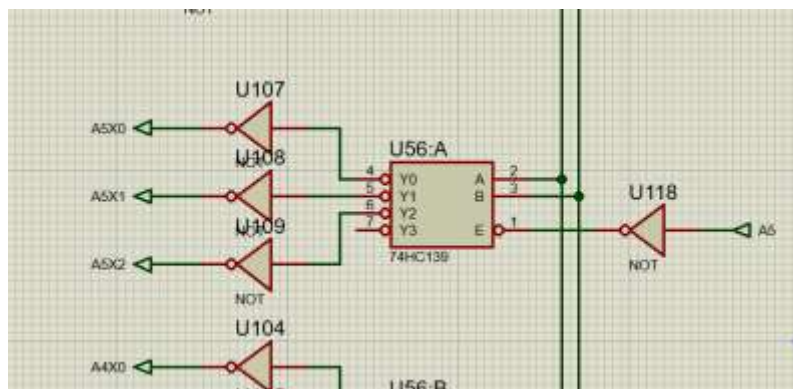


FIG: Demultiplexer

This modification transitions the standard SAP-1 ALU from a purely arithmetic unit to a multifunctional **Arithmetic Logic Unit (ALU)** by incorporating bitwise logical operations.

1. Control Logic & Signal Routing

The system uses a **Demultiplexer (74HC139)** acting as a decoder to steer the data flow based on specific control pins (Address lines A and B).

- **Input Splitting:** The 8-bit inputs are routed in parallel to three distinct sub-units:
 - **Arithmetic Unit:** Utilizing **74LS283** 4-bit Binary Full-Adders for addition and subtraction.
 - **Bitwise AND Unit:** A bank of **AND gates** (U131–U138) processing $INA \cdot INB$.
 - **Bitwise OR Unit:** A bank of **OR gates** (U139–U146) processing $INA + INB$

2. Operation Selection

The Demultiplexer serves as the operation selector. By toggling the address lines, the ALU activates only the desired computation path:

CONTROL PIN (ADDRESS)	ACTIVE OPERATION	COMPONENT
00	ARITHMATIC	74LS283
01	AND	U131-U138 AND Gates
10	OR	U139-U146 OR Gates

3. Intermediate Output Stage

Before the final result is sent to the Accumulator bus, the signals pass through an **OR-gate array (U77, U85–U91)**. These gates act as a hardware "merger," collecting the outputs from the AND, OR, and Adder blocks. Since the Demultiplexer ensures only one operation is active (logical high) at a time while others remain at high impedance or logical low, the merger outputs the result of the currently selected function without bus contention.

4. Bitwise Implementation

- **XOR Modulation:** The XOR gates (U36–U39) remain integrated with the 74LS283 units to facilitate Two's Complement subtraction when the SUB control line is active.
- **Parallel Processing:** All logic operations are performed asynchronously; the Demultiplexer simply determines which result is gated through to the output.

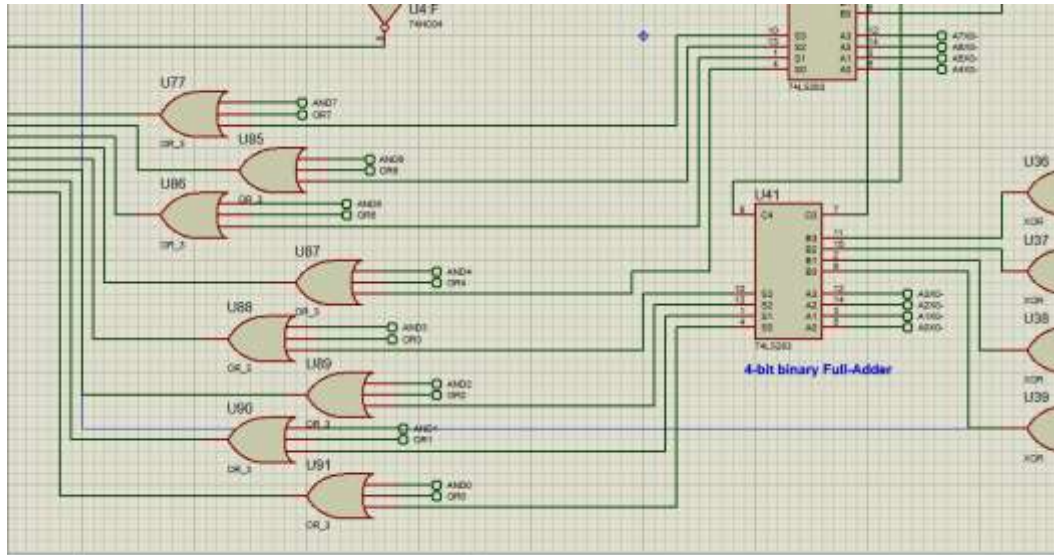


FIG: adding the 3 operations using OR gate

MICROCODE:

INSTRUCTION	OPCODE8	T-STATE	CONTROL PINS ON
AND	0100	T2	IO MI
		T3	RO BI
		T4	EO AI AN
OR	0101	T2	IO MI
		T3	RO BI
		T4	EO AI OR
MVI	0110	T2	IO AI